Exploring the combination of AI and formal methods to develop robust lock-free programs

Remi Chassagnol

NIST
National Institute of Standards and Technology
U.S. Department of Commerce

ITL

Introduction

- Context:
  - Modern parallel programs use lock-free algorithm to scale with a high number of threads.
  - Lock-free program are hard to write and test:
    - Their execution is non-deterministic.
  - LLMs are good at writing small lock-free programs, but they start making mistakes when moving to a larger scale.
- Objective:
  - Use formal methods to mathematically prove the correctness of lock-free programs (with respect to the specifications).
  - Automate a part of this work using LLMs.
- Goal of this presentation:
  - Introduction to lock-free programming and formal methods.
  - AI integration case study.

Overview

- Orchestrate the computation.
- Prevent data-races on shared memory:

```
1  bool pop(Queue *queue, Data *result) {
2      if (queue->head >= queue->tail) return false;
3      *result = queue->data[queue->head & queue->mask];
4      queue->head += 1;
5      return true;
6  }
```

- If pop is called by 2 threads in parallel:
  - It can work!
  - But threads may also pop the same element.
  - Or one the the threads may read invalid data.
  - The result is non-deterministic.

- How to solve the problem?
  - Mutual Exclusion (Mutex):

```
1  bool pop(Queue *queue, Data *result) {
2      mutex_lock(&queue->mutex); // 1 thread get the lock, the others wait
3      if (queue->head >= queue->tail) {
4          mutex_unlock(&queue->mutex); // don't leave with the lock!
5          return false;
6      }
7      *result = queue->data[queue->head & queue->mask];
8      queue->head += 1;
9      mutex_unlock(&queue.mutex); // release the lock and allow another thread
10                                 // to access the queue
11     return true;
12 }
```

- Mutexes are great in some cases:
  - Easy to use (mostly…).
  - The scaling is predictable.
  - Useful when threads need to sleep.
    - Condition variable.
- They become too slow at high thread count:
  - Create a sequential region in a parallel code.
  - The thread holding the lock can be de-scheduled by the OS.
  - Syscalls introduce latency.

- CPU instructions (user space only / no syscall).
- Allow fine grained memory ordering control (Out of Order execution).
- Only operate on register size data (32/64 to 128 bits on some machines).
- Limited to specific operations:
  - **Load**: read a value.
  - **Store**: write a value.
  - **FAX**: read the value at the given address and execute the X operation (Fetch And Add, Fetch And Sub, Fetch And Xor, …).
  - **CAS**: Compare And Swap (conditional update).
  - …

```
1    atomic_fetch_add(&queue->tail, 1);
```

- Atomic operation support explicit memory ordering (on some platform).
- This gives control on which order the CPU is allowed to execute operations:
  - CPUs don't execute operations "*line by line*", they can execute several instructions in parallel.

```
1  // Thread 1:
2  *shared_data = 42;
3  atomic_store_explicit(ready_flag, true, memory_order_release);
4
5  // Thread 2:
6  if (atomic_load_exlicit(ready_flag, memory_order_acquire)) {
7      assert(*shared_data == 42); // Guaranteed to be true!
8  }
```

```c
#include <stdatomic.h>
#include <stdbool.h>

unsigned shared_data = 0;
_Atomic unsigned ready_flag = 0;

void producer() {
    shared_data = 42;
    // what is the difference between memory order relaxed and release?
    atomic_store_explicit(&ready_flag, 1, memory_order_relaxed);
}
```

- Compile with `armv2-a clang` (with `-O2`)
  - We use `arm` because `x86-64` doesn't fully support memory ordering.
- `llvm-mca`
  - Machine Code Analyzer.
  - Simulates a particular CPU (`neoverse-v2`).
  - `-timeline` view.

D: dispatched (enqueued), e: executing, E: executed (wait), R: retired (dequeued), =: stalled in the execution queue, -: stalled in the retirement queue.

```
1  DeER .    adrp x8, shared_data              ; load shared_data page address
2  DeER .    mov  w9, #42                      ; prepare w9
3  DeER .    adrp x10, ready_flag              ; load ready_flag page address
4  DeER .    mov  w11, #1                      ; prepare w11
5  D=eER.    str  w9, [x8, :lo12:shared_data]  ; write shared_data
6  .DeER.    str  w11, [x10, :lo12:ready_flag] ; write ready_flag
7  .DeER.    ret
```

```
1  DeER .    adrp x8, shared_data              ; load shared_data page address
2  DeER .    mov  w9, #42                      ; prepare w9
3  DeER .    mov  w10, #1                      ; prepare w10
4  D=eER.    str  w9, [x8, :lo12:shared_data]  ; write shared_data
5  DeE-R.    adrp x8, ready_flag               ; load ready_flag page address
6  .DeER.    add  x8, x8, :lo12:ready_flag     ; ready_flag address (for stlr)
7  .D=eER    stlr w10, [x8]                    ; write ready flag
8  .DeE-R    ret
```

relaxed top / release bottom.

- Use atomic operations instead of mutexes.
  - Mostly lock-free algorithms still use mutexes in some cases.
- Lock-free does not necessarily means wait-free.
  - Retry loops / CAS loop.
- Guaranties that at least one thread advances its work.
  - No thread can be de-scheduled while holding a lock.
- Threads must help each other instead of blocking each other.

Micheal-Scott lock free-queue (requires 128 bits instructions).

```
1  struct NodePtr { Node *ptr; u64 count; }; // pointer + version number (tagged pointer, 16 bytes long)
2  struct Node { Data data; NodePtr next; }; // [data1,-]->[data2,-]->[data3,-]->NULL
3                                            // ^-head                  ^- tail
4  void msq_push(MS_Queue *queue, Data data) {
5      Node *node = new_node(data);
6      NodePtr tail, next;
7
8      for (;;) { // retry loop
9          tail = atomic_load16(&queue->tail);
10         next = atomic_load16(&tail.ptr->next);
11
12         if (tail == atomic_load16(&queue->tail)) {
13             if (next.ptr == NULL) { // check that the tail points to the true last node
14                 NodePtr new_next = {node, next.count + 1};
15                 if (atomic_cas16(&tail.ptr->next, next, new_next)) { // try to swap the next
16                     break; // success: node is linked
17                 } else {
18                     mm_pause(); // add some delay on failure (improves global performance)
19                 }
20             } else {
21                 // another thread poped a node, but didn't have time to update the tail yet
22                 // we help it by trying to update the tail ourselve and gain some time
23                 NodePtr new_tail = {next.ptr, tail.count + 1};
24                 atomic_cas16(&queue->tail, tail, new_tail);
25                 mm_pause();
26             }
27         }
28     }
29     // try to advance the tail after push
30     atomic_cas16(&queue->tail, tail, {node, tail.count + 1})
31 }
```

- Lock free algorithms are hard to write:
  - Low level:
    - Memory management.
    - Minimize heavy atomic operations.
    - Optimize memory layouts (padding!).
    - Think about memory ordering.
  - Thinking in parallel.
  - Need to evaluate every cases:
    - ABA safety.
- They are also hard to test:
  - Non-deterministic execution.
- AI can help, but only up to a certain point.

How to reliably verify those algorithms?

Example from the introduction of "*Program* = *Proof*" from Samuel Mimram:

- One day, a mathematician finds out that $\int_0^\infty \frac{sin(t)}{t} dt = \frac{\pi}{2}$
- then that $\int_0^\infty \frac{sin(t)}{t} \frac{sin(t/101)}{t/101} dt = \frac{\pi}{2}$
- then that $\int_0^\infty \frac{sin(t)}{t} \frac{sin(t/101)}{t/101} \frac{sin(t/201)}{t/201} dt = \frac{\pi}{2}$
- He concludes that: $\int_0^\infty (\prod_{i=0}^{n} \frac{sin(t/(100n+1))}{t(100n+1)}) dt = \frac{\pi}{2}$

Doing the actual proof will show that his conclusion is not true for every n.

- The same idea can be applied to programs:
  - Unit tests only tests for a few values of n.
  - The proof of the program assert its correctness in every cases (with respect to the specifications).
  - The proof is not impacted by the non-determinism of the execution.
- Why don't we prove every program?
  - Formal proving is difficult, and expensive.
- Proof are already done by compilers:
  - The language type system is a form of proof.
  - Optimizers build proves.
  - Some languages like Haskell or Rust use their type system to ensure memory safety or even program correctness in some cases.
- Existing formally proved software:
  - **seL4**: microkernel operating system.
  - **CompCert C**: C compiler.
  - Nearly every official cryptographic algorithms implementations.

- Programs used by mathematicians to write proofs that are checked automatically.
  - Coq (Rocq), Lean, Isabelle, Agda, ...
- expMath (Exponentiating Mathematics):
  - DARPA funded project.
  - Use formal provers with AI in Mathematics research.
- Workflow:
  - Define theorems and lemmas.
  - Use tactics defined by the language to prove them.
- Example: numbers with Coq.

Inductive definition of natural number:

```
1  Inductive nat : Set :=
2      | O : nat              (* atomic element O *)
3      | S : nat -> nat.      (* a function S(nat) -> nat (Successor) *)
```

- Coq is a functional language:
  - "*Functions are values*" (lambda calculus).
- Example:
  - $0 = O$
  - $1 = S(O)$ *(1 is the successor of 0)*
  - $2 = S(S(O))$ *(2 is the successor of 1)*
  - ...

Defining the addition:

```
1  Fixpoint plus (a b: nat) :=
2      match a with
3          | O => b
4          | S a' => S (plus a' b)
5      end.
```

- `Fixpoint` create a recursive definition.
- Pattern matching on `a`.
- `plus` is a recursive function.

Interactive proving (Example 1):

```
Theorem plus_0_n : forall n, plus 0 n = n.
Proof.
simpl.
reflexivity.
Qed.
```

```
1 goal
_____________________________________(1/1)
forall n : nat, plus 0 n = n
```

Interactive proving (Example 1):

```
Theorem plus_0_n : forall n, plus 0 n = n.
Proof.
simpl.
reflexivity.
Qed.
```

```
1 goal
_____________________________________(1/1)
forall n : nat, plus 0 n = n
```

Following the definition of `plus`, a is 0, so we can unfold the function and simplify.

```
Theorem plus_0_n : forall n, plus 0 n = n.
Proof.
simpl.
reflexivity.
Qed.
```

```
1 goal
_____________________________________(1/1)
forall n : nat, n = n
```

Interactive proving (Example 1):

```
Theorem plus_0_n : forall n, plus 0 n = n.
Proof.
simpl.
reflexivity.
Qed.
```

```
1 goal
_____________________________________(1/1)
forall n : nat, plus 0 n = n
```

Following the definition of `plus`, a is 0, so we can unfold the function and simplify.

```
Theorem plus_0_n : forall n, plus 0 n = n.
Proof.
simpl.
reflexivity.
Qed.
```

```
1 goal
_____________________________________(1/1)
forall n : nat, n = n
```

n = n is proved by reflexivity.

```
Theorem plus_0_n : forall n, plus 0 n = n.
Proof.
simpl.
reflexivity.
Qed.
```

```
All goals completed.
```

Interactive proving (Example 2):

```
Theorem plus_n_0 : forall n, plus n 0 = n.
Proof.
intros n.
```

```
1 goal
_____________________________________(1/1)
forall n : nat, plus n 0 = n
```

```
Theorem plus_n_0 : forall n, plus n 0 = n.
Proof.
intros n.
```

```
1 goal
_____________________________________(1/1)
forall n : nat, plus n 0 = n
```

Easy, we can just simplify right?

```
Theorem plus_n_0 : forall n, plus n 0 = n.
Proof.
simpl.
```

```
1 goal
_____________________________________(1/1)
forall n : nat, plus n 0 = n
```

`plus` matches a, and a is not 0 in this case, we need an induction proof.

```
1 Fixpoint plus (a b: nat) :=
2     match a with
3         | O => b
4         | S a' => S (plus a' b)
5     end.
```

```
Theorem plus_n_0 : forall n, plus n 0 = n.
Proof.
intros n.
```

```
1 goal
______________________________________(1/1)
forall n : nat, plus n 0 = n
```

The `intros` command introduces a `n` in the proof (removes the `forall`).

```
Theorem plus_n_0 : forall n, plus n 0 = n.
Proof.
intros n.
induction n as [| n'].
```

```
1 goal
n : nat
______________________________________(1/1)
plus n 0 = n
```

We start the induction proof on n, and we end up with 2 goals: the base case, and the recursion case.

```
Theorem plus_n_0 : forall n, plus n 0 = n.
Proof.
intros n.
induction n as [| n'].
auto.
```

```
2 goals
_____________________________________(1/2)
plus 0 0 = 0
_____________________________________(2/2)
plus (S n') 0 = S n'
```

Coq will solve the sub-goals in order, the first one can be resolve automatically using `auto`. We then end up on the recursion case and Coq generates the induction hypothesis on n (`IHn'`).

```
Theorem plus_n_0 : forall n, plus n 0 = n.
Proof.
intros n.
induction n as [| n'].
auto.
```

```
1 goal
n' : nat
IHn' : plus n' 0 = n'
_____________________________________(1/1)
plus (S n') 0 = S n'
```

`simp` will apply the definition of `plus` and simplify.

```
Theorem plus_n_0 : forall n, plus n 0 = n.
Proof.
intros n.
induction n as [| n'].
auto.
simpl.
```

```
1 goal
n' : nat
IHn' : plus n' 0 = n'
______________________________________(1/1)
S (plus n' 0) = S n'
```

We can then `rewrite` the hypothesis inside the goal.

```
Theorem plus_n_0 : forall n, plus n 0 = n.
Proof.
intros n.
induction n as [| n'].
auto.
simpl.
rewrite IHn'.
reflexivity.
Qed.
```

```
1 goal
n' : nat
IHn' : plus n' 0 = n'
______________________________________(1/1)
S n' = S n'
```

- Using the Iris framework:
  - Higher-order concurrent separation logic framework.
  - Introduces a custom language called Heap-lang.
    - Model program interactions with the heap (resource algebra).
  - Adds new tactics.
  - Models side effects (ghost state).
  - Supports parallelism.
- Workflow:
  1. Express program interactions with the memory in heap-lang (equivalent to translating the original program to heap-lang).
  2. Write program specifications, which usually take the form of Hoare triples: $S \vdash \{P\}e\{v.Q\}$.
  3. Complete the prof using tactics.

Resource algebra notations:

- Entailment ($\vdash$):
  - Implication for resources.
  - $P \vdash Q$: the resource P can be transformed into the resource Q.
- Separating conjunction ($*$): equivalent to a logic and ($P * Q$).
- Magic wand ($-\!\!*$):
  - $P -\!\!* Q$: If I'm given a resource P, I can produce the resource Q.
  - Modus ponens: $P * (P -\!\!* Q) \vdash Q$.
- Later modality ($\triangleright P$):
  - P will be true after the next step of computation.
  - Löb Induction: $(\triangleright P -\!\!* P) \vdash P$
    - If I can prove P assuming $\triangleright P$, them P is always true.
    - Useful when proving loops.

Resource algebra notations:

- Hoare triples ($S \vdash \{P\}e\{v.Q\}$):
  - A specification S implies that from a set of preconditions P, the expression e evaluates safely and gives a value v and a set of post conditions Q.
- Rules:
  - $\frac{Premise_1 \ldots Premise_n}{conclusion}\, rule\_name.$

$$\frac{R * P \vdash Q}{R \vdash P \mathrel{-\!\!*} Q}$$

If with the resource R and the resource P I can produce the resource Q, then having the resource R implies that if I'm given the resource P, I can produce the resource Q.

$$\frac{R_1 \vdash P \mathrel{-\!\!*} Q \qquad R_2 \vdash P}{R_1 * R_2 \vdash Q}$$

Having R_1 implies that if I have P I can get Q, and having R_2 implies that I have P, then having R_1 and R_2 implies that I can get Q.

Proof tree:

$$\dfrac{\dfrac{\dfrac{P * Q \vdash P * Q}{P * Q \vdash P * Q \vee P * R}}{Q \vdash P \mathrel{-\!\!*} (P * Q \vee P * R)} \quad \dfrac{\dfrac{P * R \vdash P * R}{P * R \vdash P * Q \vee P * R}}{R \vdash P \mathrel{-\!\!*} (P * Q \vee P * R)}}{\dfrac{Q \vee R \vdash P \mathrel{-\!\!*} (P * Q \vee P * R)}{P * (Q \vee R) \vdash P * Q \vee P * R}}$$

- Proof on paper:
  - Goal at the bottom (proved theorem).
  - Applying rules (tactics) move up a level.
  - The proof is complete once all the sub-goals are proved.

Heap lang:

```
1  Definition inc : expr :=
2    λ: "l",
3      let: "n" := !"l" in  (* load the value at the location l into n *)
4        "l" <- "n" + #1.   (* store n + 1 at the location l *)
```

- Define a function `inc`, which increments a counter.
- `"l"` is a location in memory, given as function argument.
- `!"l"` reads the value stored at l's location (equivalent to `*l` in C).
- `#1` simply means 1, and the `#` is used to specify that this is a heap lang value.
- `"l" <- "n" + #1` loads $n + 1$ into l's location (`*l = n + 1`).

Writing the specification $S \vdash \{P\}e\{v.Q\}$:

```
1  Lemma inc_spec (l: loc) (n: Z):
2    {{{ l ↦ #n }}}                    (* (weakest) precondition *)
3      inc #l                          (* expression (program) *)
4    {{{ RET #(); l ↦ #(n + 1) }}}.    (* result value; post condition *)
```

- Considering a location l pointing to a memory location that stores the value n.
- We call the `inc` function with l as argument.
- We expect no return value, and l should store the value $n + 1$.

- The *weakest* preconditions:
  - The preconditions that describes the highest amount of values.
- Example:
  - $n \in [1; 10]$ is strong.
  - $n \in \mathbb{N}$ is weaker.
  - $n \in \mathbb{Z}$ is even weaker.

```
Lemma inc_spec (l: loc) (n: Z):
  {{{ l ↦ #n }}}
    inc #l
  {{{ RET #(); l ↦ #(n + 1) }}}.
Proof.
  iIntros (Φ) "Hl Hcl".
  wp_pures.
  wp_load.
  wp_pures.
```

```
1 goal
Σ : gFunctors
heapGS0 : heapGS Σ
spawnG0 : spawnG Σ
N : namespace
l : loc
n : Z
_____________________________________(1/1)
{{{ l ↦ #n }}} inc #l {{{ RET #(); l ↦ #(n + 1) }}}
```

We start by introducing a state ϕ, and we name the precondition and post condition Hl and Hcl.

```
Lemma inc_spec (l: loc) (n: Z):
  {{{ l ↦ #n }}}
    inc #l
  {{{ RET #(); l ↦ #(n + 1) }}}.
Proof.
  iIntros (Φ) "Hl Hcl".
  wp_pures.
  wp_load.
  wp_pures.
  wp_store.
  iApply "Hcl".
  iApply "Hl".
Defined.
```

```
1 goal
Σ : gFunctors
heapGS0 : heapGS Σ
spawnG0 : spawnG Σ
N : namespace
l : loc
n : Z
Φ : val → iPropI Σ
------------------------------------(1/1)
"Hl" : l ↦ #n
"Hcl" : ▷ (l ↦ #(n + 1) -∗ Φ #())
------------------------------------∗
WP inc #l {{ v, Φ v }}
```

wp_pures acts on the code, here it will unfold the function, and simplify if possible.

```
Lemma inc_spec (l: loc) (n: Z):
  {{{ l ↦ #n }}}
    inc #l
  {{{ RET #(); l ↦ #(n + 1) }}}.
Proof.
  iIntros (Φ) "Hl Hcl".
  wp_pures.
  wp_load.
  wp_pures.
  wp_store.
  iApply "Hcl".
  iApply "Hl".
Defined.
```

```
1 goal
Σ : gFunctors
heapGS0 : heapGS Σ
spawnG0 : spawnG Σ
N : namespace
l : loc
n : Z
Φ : val → iPropI Σ
_____________________________________(1/1)
"Hl" : l ↦ #n
"Hcl" : l ↦ #(n + 1) -∗ Φ #()
_____________________________________∗
WP let: "n" := ! #l in #l <- "n" + #1 {{ v, Φ v }}
```

We can run the first instruction which is a load (and a let which will be simplified).

The load and the let have been executed, and the expression was simplified.

```
Lemma inc_spec (l: loc) (n: Z):
  {{{ l ↦ #n }}}
    inc #l
  {{{ RET #(); l ↦ #(n + 1) }}}.
Proof.
  iIntros (Φ) "Hl Hcl".
  wp_pures.
  wp_load.
  wp_pures.
  wp_store.
  iApply "Hcl".
  iApply "Hl".
Defined.
```

```
1 goal
Σ : gFunctors
heapGS0 : heapGS Σ
spawnG0 : spawnG Σ
N : namespace
l : loc
n : Z
Φ : val → iPropI Σ
_____________________________________(1/1)
"Hl" : l ↦ #n
"Hcl" : l ↦ #(n + 1) -∗ Φ #()
-------------------------------------∗
WP #l <- #(n + 1) {{ v, Φ v }}
```

Then, we can execute the store instruction (<-).

The store was executed, changing the value of l. The Hl hypothesis was updated.

```
Lemma inc_spec (l: loc) (n: Z):
  {{{ l ↦ #n }}}
    inc #l
  {{{ RET #(); l ↦ #(n + 1) }}}.
Proof.
  iIntros (Φ) "Hl Hcl".
  wp_pures.
  wp_load.
  wp_pures.
  wp_store.
  iApply "Hcl".
  iApply "Hl".
Defined.
```

```
1 goal
Σ : gFunctors
heapGS0 : heapGS Σ
spawnG0 : spawnG Σ
N : namespace
l : loc
n : Z
Φ : val → iPropI Σ
_____________________________________(1/1)
"Hl" : l ↦ #(n + 1)
"Hcl" : l ↦ #(n + 1) -∗ Φ #()
_____________________________________∗
|={⊤}=> Φ #()
```

- There is no more code, we are left with the final state ϕ `#()`.
- Hcl is what we are trying to prove. It says that the final state should be `l -> #(n + 1)`.

Applying Hcl expends the value of ϕ `#()`, this is the final goal:

```
Lemma inc_spec (l: loc) (n: Z):
  {{{ l ↦ #n }}}
    inc #l
  {{{ RET #(); l ↦ #(n + 1) }}}.
Proof.
  iIntros (Φ) "Hl Hcl".
  wp_pures.
  wp_load.
  wp_pures.
  wp_store.
  iApply "Hcl".
  iApply "Hl".
Defined.
```

```
1 goal
Σ : gFunctors
heapGS0 : heapGS Σ
spawnG0 : spawnG Σ
N : namespace
l : loc
n : Z
Φ : val → iPropI Σ
--------------------------------------(1/1)
"Hl" : l ↦ #(n + 1)
--------------------------------------*
|={⊤}=> l ↦ #(n + 1)
```

The final goal is proved by Hl, applying it finishes the proof.

Parallelism:

```
1  (* define a counter with a simplified notation *)
2  Definition counter (l: loc): expr := #l <- !#l + #1.
3
4  (* parallel programs require invariants *)
5  Definition counter_inv (l: loc) (n: Z): iProp :=
6    (∃ (m: Z), ⌜(n ≤ m)%Z⌝ ∗ l ↦ #m)%I.
7
8  (* parallel counter specification *)
9  Lemma parallel_counter_spec (l: loc) (n: Z):
10   {{{ l ↦ #n }}}
11     (counter l) ||| (counter l) ;; !#l  (* ||| parallel execution ;; return value *)
12   {{{ m, RET #m; ⌜(n ≤ m)%Z⌝ }}}.
13 Admitted.
```

- Proving $n \leq m$ is easy, but we could prove that $m = n + 1 \vee m = n + 2$.
- We cannot prove $m = n + 2$ because the operation is not atomic:
  - Would require using a lock or FAA.

```
Lemma parallel_counters_spec (l: loc) (n: Z):
  {{{ l ↦ #n }}}
    (counter l) ||| (counter l) ;; !#1
  {{{ m, RET #m; ⌜(n ≤ m)%Z⌝ }}}.
Proof using N heapGS0 spawnG0 Σ.
  iIntros (Φ) "Hl Hcl". (* ⊢ intros hypothesis *)
  rewrite /counter. (* apply counter definition *)
  iMod (inv_alloc N _ (counter_inv l n) with "[Hl]") as "#HInv". (* Allocate the shared global invariant *)
  { iExists n. iFrame. eauto with lia. } (* prove H1 *)
  wp_pures.
  wp_apply (wp_par (λ _, True)%I (λ _, True)%I with "[] []"). (* move both sides of the par to subgoals *)

  - (* --- Left Thread --- *)
    wp_bind (!#1)%E. (* the current WP denpends on !#1, so we need to bind it to a value v *)

    (* to open the invariant, we can either use both iInv and iDestruct*)
    (*
    iInv N as "H" "Hclose". (* open the invariant *)
    iDestruct "H" as (m) ">[% Hpt]". (* eliminate existential qualifiers in the hypothesis: > removes the later modality, Hfp is the new hypothesis, % extract the pure facts *)
    *)
    (* or use iInv directly *)
    iInv N as (m) ">[% Hpt]" "Hclose".

    wp_load. (* run the load *)
    iMod ("Hclose" with "[Hpt]") as "_". (* use iMod to remove the modality and close the invariant *)
    { iNext; (* remove the later modality to get the invariant back on the first sub-goal *)
      iExists m; (* intros m in the invariant *)
      iFrame; (* we can use iFrame to remove the l -> #m since it is Hfp *)
      iIntros "!%"; (* extract pure context from the expression to remove the assert and Move to the Coq level *)
      apply H. (* H tells us that the expression is true, we can move on *)
    }
    iMooIntro.
    wp_pure.
    iInv N as (m') ">[% Hpt]" "Hclose".
    wp_store.
    iMod ("Hclose" with "[Hpt]") as "_".
    { iNext. iFrame. iIntros "!%". by lia. }
    auto.

  - (* --- Right Thread --- *)
    wp_bind (!#1)%E. (* the current WP denpends on !#1, so we need to bind it to a value v *)
    iInv N as "H" "Hclose". (* open the invariant *)
    iDestruct "H" as (m) ">[% Hpt]". (* eliminate existential qualifiers in the hypothesis: > removes the later modality, Hfp is the new hypothesis, % extract the pure facts *)
    wp_load. (* run the load *)
    iMod ("Hclose" with "[Hpt]") as "_". (* use iMod to remove the modality and close the invariant *)
    { iNext; (* remove the later modality to get the invariant back on the first sub-goal *)
      iExists m; (* intros m in the invariant *)
      iFrame; (* we can use iFrame to remove the l -> #m since it is Hfp *)
      iIntros "!%"; (* extract pure context from the expression to remove the assert and Move to the Coq level *)
      apply H. (* H tells us that the expression is true, we can move on *)
    }
    iMooIntro.
    wp_pure.
    iInv N as (m') ">[% Hpt]" "Hclose".
    wp_store.
    iMod ("Hclose" with "[Hpt]") as "_".
    { iNext. iFrame. iIntros "!%". by lia. }
    auto.

  - (* return value *)
    iIntros.
    iNext.
    wp_pures.
    iInv N as (m) ">[% Hpt]" "Hclose".
    wp_load.
    iMod ("Hclose" with "[Hpt]") as "_".
    { by iFrame. } (* equivalent to: iNext. iExists m. iFrame. iIntros "!%". auto. *)
    iMooIntro.
    iApply "Hcl".
    iPureIntro. done.
Qed.
```

- These are simple proofs.
- Heap-lang can do more:
  - Allocate memory (`AllocN`).
  - Compute offsets (assign 4 to the second field of obj: `"obj" +ₗ #1 <- #4`).
  - Atomic operations.
  - …
- There is more to learn about the tactics.
- There is more theory:
  - Invariants.
  - Ghost state.
  - …

- Writing the actual proof is a mechanical exercise:
  - Applying tactics till the proof is done.
  - The theorem prover ensure the correctness of the proof.
- Workflow:
  1. Write the code (manually or using an agent).
  2. Use an agent to port the code to heap-lang (first potential point of failure).
  3. Write the specifications:
     - AI will translate english into Coq.
  4. Use the AI to write the proof and report on failure.

- Agents reading the code must be told not to try to fix it (heap-lang port).
  - The agent will have limited context (no access to the whole application).
  - The proven code is expected to be complex, even good models make mistakes in those cases.
- The proof must be done interactively:
  - The AI should not try to "*one-shot*" the proof.
  - Use MCP/LSP servers.
  - Writing proofs is a long and difficult process.
- Writing programs specifications is difficult.
- Maintain the program $\Rightarrow$ maintain the proof.

- Experiment:
  1. Use the agent to implement a nearly lock-free chunked queue.
     - The algorithm is simple.
     - A subtile bug will be introduced in the implementation.
  2. Translate the source code to Heap lang.
  3. Write the specifications.
  4. Build the proofs.
- Model: Opus 4.6
  - More models will be tested later.

```
chunked_queue_push :: proc(queue: ^Chunked_Queue($T), data: T) {
	pos := sync.atomic_add_explicit(&queue.tail_idx, 1, .Release)
	block_id := u64(pos / CHUNKED_QUEUE_BLOCK_SIZE)
	slot_idx := pos % CHUNKED_QUEUE_BLOCK_SIZE

	if slot_idx == 0 && pos > 0 {
		sync.mutex_lock(&queue.grow_mutex)
		tail := sync.atomic_load_explicit(&queue.tail_block, .Relaxed)
		for tail.block_id < block_id {
			next := tail.next
			if sync.atomic_load_explicit(&next.empty_count, .Acquire) == CHUNKED_QUEUE_BLOCK_SIZE {
				sync.atomic_store_explicit(&next.empty_count, 0, .Release)
				next.block_id = tail.block_id + 1
				tail = next
			} else {
				new_block, _ := new(Chunked_Queue_Block(T))
				new_block.block_id = tail.block_id + 1
				new_block.next = tail.next
				sync.atomic_store_explicit(&tail.next, new_block, .Release)
				tail = new_block
				queue.capacity += CHUNKED_QUEUE_BLOCK_SIZE
			}
		}
		sync.atomic_store_explicit(&queue.tail_block, tail, .Release)
		sync.mutex_unlock(&queue.grow_mutex)
	}

	for {
		cursor := sync.atomic_load_explicit(&queue.tail_block, .Acquire)
		start := cursor
		for {
			if cursor.block_id == block_id {
				slot := &cursor.data[slot_idx]
				sync.atomic_store_explicit(&slot.state, .Writing, .Relaxed)
				slot.value = data
				sync.atomic_store_explicit(&slot.state, .Valid, .Release)
				return
			}
			cursor = cursor.next
			if cursor == start { break }
		}
		sync.cpu_relax()
	}
}
```

```
# Porting the code to heap-lang

We are proving a program using `Coq` and the `Iris` framework. Your role is
to translate source code into Iris `heap_lang`.

## Rules

- Ignore comments.
- Do not analyze the code, focus on the translation. If there are bugs, they
  will be found through the proof so port the code exactly as it is, otherwise
  the final proof will be wrong.
- For the proof to be valid, the `heap_lang` code MUST match the source code
  perfectly.
- Use recursion to model the loops.
- Split the complex loops into different definitions.
- Do not write any proofs or specifications, output only the `heap_lang`
  definitions using the `val` type.
- Use `rocq-mcp` and `coq-lsp` to verify the Coq files.
```

- This prompt will change depending on the project:
  - Big projects may already have existing models for locks, and other data structures.
  - Organizing the files and telling the LLM where to look can save context.
- Even when porting simple code, the model can make mistakes.
  - Double check second pass.
  - Emphasize the importance of a perfect match in the prompt.

- We will use generic prompts and put the application specific details directly in the code.
  - `@spec`: general specifications.
  - `@inv`: invariant (including ownership invariant).
  - `@linpoint`: atomic updates.

```
1  // @inv(Queue State):
2  // - Ghost state: An authoritative append-only list `L` representing the
3  //                logical history of all pushed elements.
4  // - `tail_idx` is a monotonically increasing ticket counter. Its value equals
5  //   the length of `L`.
6  // - `head_idx` is a monotonically increasing ticket counter representing the
7  //   number of popped elements.
8  Chunked_Queue :: struct($T: typeid) #align(64) { ... }
```

- A generic specification will not find small bugs.
- Ownership and ghost state specification are important.

```
// @inv(Slot Ownership):
// The state of the slot dictates memory permissions in Separation Logic:
// - Empty: The queue invariant owns the state. No thread has permission to
//          read/write `value`.
// - Writing: The producer thread holds exclusive permission (`value ↦ _`) to
//            write. The queue invariant does NOT own `value`.
// - Valid: The producer has relinquished permission. The queue invariant now
//          owns `value ↦ data`, and `data` maps to the corresponding logical
//          index in the ghost state.
Slot_State :: enum {
        Empty,   // Ready for a producer to claim
        Writing, // Producer is writing (do not read)
        Valid,   // Data is written and ready for consumer
}
```

- General specifications will allow reusing the queue in bigger proofs:
  - A program doing a queue push will use the push spec instead of re-proving the whole queue code.

```
1  //
2  // @spec: logically atomic push that appends `data` to the ghost state list `L`.
3  // @linpoint: The atomic FAA on `queue.tail_idx`.
4  //
5  chunked_queue_push :: proc(queue: ^Chunked_Queue($T), data: T) {
6      ...
7  }
```

```
# Writing the specification

We are proving a program using `Coq` and the `Iris` framework. Based on the
`@spec`/`@inv`/`@linpoint` comments in the source code and the `heap_lang`
implementation, generate the `Iris` specifications:
1. Ghost State: Define the necessary Resource Algebras (Cameras) to represent
   the logical state described in the comments.
2. Invariants: Define the persistent internal invariant (e.g. IsData). You
   MUST strictly enforce the memory ownership rules described in the `@inv`
   comments, linking physical state (enums/flags) to Separation Logic
   permissions (like l ↦ v).
3. Logically Atomic Specs: Write the Hoare triples for the public functions
   using the Logically Atomic notation (<<< ∀ state, ... >>>) at the execution
   points defined by `@linpoint`.

Leave the proofs `Admitted`.

## Rules
...
```

- Specifications should be precise:
  - General specifications (reusable)
  - Mapping between logical state and its separation logic ownership (slot state / ownership of the data).
  - Bugs hides in the details:
    - A subtly bugged push will work 90% of the time, only precise resource analysis can prove the bug.
- The proof only proves the specifications, nothing more...
- Writing the specifications is the hardest part of the process.

```
# Writing the proof

We are proving a program using `Coq` and the `Iris` framework. Your role will
be to complete `Admitted` proofs using `Coq` and `Iris` strategies. If a proof
is impossible due to a code bug, explain exactly which spatial resource is
missing and at which instruction the verification fails.

## Rules

- Do not change the specifications or the invariant definition.
- Do not read or change the source code.
- You can read the `heap_lang` implementation but you should not try to find
  bugs from here. If there are any issues with the program, they should be
  found through the proof.
- Do not try to write the entire proof in "one shot": use `rocq-mcp` and
  `coq-lsp` to complete the proof **interactively**. At each step, refer to
  Coq's output to build the next step of the proof.
- Use `Defined` instead of `Qed`.

...
```

- When there are complicated proofs, the LLM should be focused on a single one.
  - Avoid context explosion.
- Break complex specifications into small proofs:
  - Reduce context.
  - Otherwise the AI will *fight* Coq (struggle with complex Coq features).
- Having a cheat-sheet/skill that includes techniques to solve proofs with Coq may help the AI.
- LLMs like to find holes in the rules:
  - "*The rules says that I can't change the specifications, but I can change the implementation...*", Opus 4.6
- Writing the proof takes time:
  - Mechanical yet complex process.
  - Issues with the specifications (notably invariants) make proving even longer.
  - Theorem provers can be difficult to use.

The LLM got stuck and reported:

Before proceeding further with this proof, I need to analyze the code carefully. Looking at chunked_queue_push_find_block:

```
"slot" +ₗ #1 <- #1;;   (* state := Writing *)
"slot" +ₗ #1 <- #2;;   (* state := Valid *)
"slot" <- "data";;      (* write data *)
```

This is a bug in the program. The writes are in the wrong order.

According to the slot_inv invariant:
- state=0 (Empty): invariant owns both state and value (slot_loc ↦ #0)
- state=1 (Writing): producer holds exclusive value permission; invariant owns state only
- state=2 (Valid): invariant owns state AND value with slot_loc ↦ v where L !! pos = Some v

The transition from state 1→2 requires the producer to surrender the value cell back to the invariant, with the correct data already written. But in the code, state is set to 2 before the data is written to the value cell.

Specifically: At the instruction "slot" +ₗ #1 <- #2, verification fails because:
- The producer holds slot_loc ↦ #0 (the initial zero, data not yet written)
- To close the invariant with state=2, we need ∃ v, slot_loc ↦ v ∗ ⌜L !! pos = Some v⌝
- Since L !! pos = Some data (from the FAA linearization point that appended data), we'd need slot_loc ↦ data, but we only have slot_loc ↦ #0
- The spatial resource slot_loc ↦ data is missing — we have slot_loc ↦ #0 instead

Fix: The correct instruction order should be:

```
"slot" +ₗ #1 <- #1;;   (* state := Writing — acquire value cell *)
"slot" <- "data";;      (* write data while we own the cell *)
"slot" +ₗ #1 <- #2;;   (* state := Valid — release value cell back *)
```

The chunked_queue_pop_spec likely has the same structural dependency on this ordering being correct (the consumer waits for state=2 then reads the value), so fixing push fixes pop too.

Shall I stop here, or would you like me to fix the source code and then complete both push and pop proofs?

● Rather than driving the full proof forward through the FAA and grow loop (which would be extremely long), let me demonstrate the exactfailure point more directly. I'll show it by assuming we've reached the inner chunked_queue_push_find_block body where the three slot operations happen, with the invariant open.

  The code does:
  "slot" +ₗ #1 <- #1;;   (* Empty→Writing: take value cell from invariant *)
  "slot" +ₗ #1 <- #2;;   (* Writing→Valid: give value cell back — BUT data not written yet! *)
  "slot" <- "data";;   (* write data — BUT we no longer own the cell! *)

  Let me trace through the invariant protocol at each step.

  Step 1: slot +ₗ 1 <- #1 (set state to Writing)

  We open the invariant, find slot_inv for our position with state=0:
  (slot +ₗ 1) ↦ #0 ∗ (slot ↦ #0)
  We store #1 to slot +ₗ 1, taking the value cell slot ↦ #0 and closing the invariant with state=1 (which owns only the state cell). After this, the producer holds slot ↦ #0.

  Step 2: slot +ₗ 1 <- #2 (set state to Valid)

  We open the invariant again, find slot_inv with state=1:
  (slot +ₗ 1) ↦ #1
  We store #2 to slot +ₗ 1. Now we must close with state=2, which requires:
  ∃ v, slot ↦ v ∗ ⌜L !! pos = Some v⌝
  We still hold slot ↦ #0. We'd need L !! pos = Some #0. But L !! pos = Some data (the value we're pushing).
  Unless data = #0, this is impossible.

  Let me confirm this by trying to open and close the invariant at step 2. Called rocq-mcp (ctrl+o to expand)
● Need iNext before destructing under later. Let me fix. Called rocq-mcp 3 times (ctrl+o to expand)
● The Iris destruct pattern is finicky with the nesting. Let me fix. Called rocq-mcp (ctrl+o to expand)
● Now I'm at the critical point. After wp_store set the state to 2, I need to close the invariant with ▷ slot_inv L pos slot.
  Let me show the exact failure. Called rocq-mcp (ctrl+o to expand)
● Here is the concrete proof failure:

  Goal:
  L !! pos = Some #0

  Hypothesis:
  HL : L !! pos = Some v

  where v is universally quantified (arbitrary). This goal is unprovable — we'd need v = #0, but v is the arbitrary value being pushed by the user. Called rocq-mcp (ctrl+o to expand)
● All three attempts fail. The goal L !! pos = Some #0 is unprovable given only HL : L !! pos = Some v with v arbitrary.

```
Summary of the confirmed failure:

After the code executes slot +ₗ 1 <- #2 (setting state to Valid), the invariant must be closed with the state=2 case of slot_inv,
which requires:

∃ w, slot ↦ w ∗ ⌜L !! pos = Some w⌝

At this point, the producer only holds slot ↦ #0 (the initial zero — data hasn't been written yet). The only witness we can provide is
w := #0, giving the unprovable obligation:

L !! pos = Some #0    (* but L !! pos = Some v, with v arbitrary *)

The data store slot <- data comes after the state has already been set to 2. But once state=2, the invariant takes ownership of the value
cell, so the producer can no longer write to slot
— the spatial resource slot ↦ _ has been surrendered.

The fix is to swap the last two operations: write data while in state 1 (when the producer owns the value cell), then transition to
state 2 to release it.
```

- Both the source code and the heap-lang code need to be updated.
- Continue the proving process with a fresh context.

- The final proof will be big and complex:
  - Implementation: $\sim$200 lines.
  - Proof: $>$2K lines.
- Proving that push and pop are logically atomic is difficult:
  - Parallel code proof are more difficult:
    - Logical atomicity / linearizability.
    - Ghost state and invariants.
    - Interleaving and helping (common in lock-free algorithms).
  - Proof/code ratio:
    - Sequential code: 3/1 to 5/1.
    - Parallel code: 10/1 to 20/1.

Conclusion

- Lock-free algorithms use atomic operations instead of locks.
  - Difficult to write and test.
- Formal methods:
  - Assisted proving with theorem provers.
  - Can be used to reliably prove lock-free programs.
- AI integration:
  - Automate a part of the proving process.
- Future work:
  - Test other models.
  - Try more difficult bugs.
  - Improve the prompts:
    - Try to accelerate proof writing.
    - Rules/Constrains may not be optimal: act as barriers between the model and the goal rather than being part of the goal (the model will try to break the rules to achieve the goal).

Some links

- Rocq docs (Coq): https://rocq-prover.org/docs
- Iris project: https://iris-project.org/
- Iris lecture notes: https://iris-project.org/tutorial-pdfs/iris-lecture-notes.pdf
- Coq + Iris beginner guide: https://arxiv.org/pdf/2105.12077
- A Case Study on the Effectiveness of LLMs in Verification with Proof Assistants (target more functional languages): https://arxiv.org/html/2508.18587v1
- Iris github: https://github.com/JasonGross/iris-coq
- Case study source code: https://github.com/drfailer/lock_free_program_proving

Thanks !